

HAUQE CERTIF

Guide d'administration

des règles, de la codification, du scoring et des grilles de contrôle FUCCS

BNEC / SNGSCC - Document opérationnel utilisateur

Version	1.0
Date	28 juillet 2026
Public cible	Administrateurs habilités HAUQE
Périmètre	COLLECTE_COMPLETUDE, règles métier, scoring, INFC, FUCCS

Principe directeur : aucune règle n'est auto-publiée. Toute version exécutable doit être approuvée, datée et auditée.

0 Comment utiliser ce guide

Ce document accompagne l'onglet « Règles et codification » de HAUQE Certif. Il décrit les écrans, les conventions de code, les JSON attendus, les contrôles avant publication et la manière de faire évoluer une version sans casser l'historique.

Important

Les exemples signalés « à valider » servent de modèles techniques. Ils ne deviennent pas des décisions institutionnelles tant qu'ils ne sont pas approuvés par la HAUQE et publiés avec une référence officielle.

Parcours recommandé

1 Préparer

Rassembler la décision, le code logique, la version, les paramètres et la date d'effet.

2 Créer un brouillon

Saisir ou importer la configuration sans impact sur les calculs actifs.

3 Valider

Contrôler la structure JSON, les bornes, les pondérations et les codes.

4 Tester

Utiliser la prévisualisation ou un environnement de recette.

5 Publier

Renseigner la référence d'approbation et la date d'effet.

6 Faire évoluer

Cloner la version publiée, modifier le clone, puis publier la nouvelle version.

Permissions minimales

Fonction	Lecture	Administration
Règles métier	GOUVERNANCE.LIRE	GOUVERNANCE.ADMINISTRER_REGLES
Modèles de scoring	SCORING.LIRE	SCORING.ADMINISTRER_MODELE
Grilles FUCCS	FUCCS.LIRE	FUCCS.ADMINISTRER_GRILLE

Bouton absent

Si un bouton de création ou de publication n'apparaît pas, vérifier d'abord le rôle et les permissions du compte. Ne pas contourner le contrôle côté navigateur.

Table des matières opérationnelle

- 1. Cycle de vie, versionnement et codification
- 2. Créer et publier COLLECTE_COMPLETUE
- 3. Créer une règle métier générique
- 4. Configurer les modèles de scoring et l'INFC
- 5. Créer, coder et publier une grille FUCCS
- 6. Checklists, erreurs fréquentes et annexes JSON

1

Cycle de vie et codification

Les conventions communes à tous les paramètres institutionnels.

1.1 Le cycle de vie commun

Statut	Ce qui est permis	Règle de gestion
BROUILLON	Créer, modifier, compléter, tester.	Aucun effet sur les calculs actifs.
PUBLIE	Consulter et utiliser.	Version immuable et exécutable selon sa date d'effet.
RETIRE	Consulter l'historique.	La version n'est plus utilisée pour de nouveaux traitements.

Immutabilité

Une règle, un modèle ou une grille publiée ne se corrige pas directement. Il faut la cloner, créer une nouvelle version brouillon, la tester puis la publier.

1.2 Code logique, code physique et version

Le code logique identifie le concept fonctionnel stable. La version identifie une édition. Pour les règles métier génériques, le backend fabrique automatiquement un code physique unique.

Exemple de règle versionnée

```
Code logique : COLLECTE_COMPLETUDE
Version      : 1.0
Code physique: COLLECTE_COMPLETUDE__V1_0
```

L'utilisateur travaille avec le code logique. Le code physique assure l'unicité technique dans la base.

1.3 Convention de codification

Objet	Format recommandé	Exemple
Règle métier	MAJUSCULES_ET_TIRETS_BAS	VEILLE_SEUILS_EXPIRATION
Modèle de scoring	OBJET ou OBJET_PERIMETRE	CLASSIFICATION_ENTREPRISE
Grille FUCCS	FUCCS ou FUCCS_PERIMETRE	FUCCS
Rubrique FUCCS	<GRILLE>_Rnn	FUCCS_R01
Critère FUCCS	<RUBRIQUE>_Cnn	FUCCS_R01_C01

Objet	Format recommandé	Exemple
Version	MAJEURE.MINEURE	1.0, 1.1, 2.0

- Utiliser uniquement des lettres A-Z, des chiffres et le tiret bas.
- Ne pas mettre la version dans le code des rubriques ou critères FUCCS.
- Ne pas réutiliser un code retiré pour un contenu différent.
- Conserver l'ordre d'affichage dans le champ « ordre », séparé du code.

1.4 Version mineure ou majeure ?

Type d'évolution	Version conseillée	Exemple
Correction de libellé sans changement de sens	Mineure	1.0 -> 1.1
Ajout d'un critère ou d'une exigence	Mineure ou majeure selon impact	1.1 -> 1.2 ou 2.0
Changement de formule ou de seuil institutionnel	Majeure	1.x -> 2.0
Nouvelle famille fonctionnelle	Nouveau code logique	FUCCS -> FUCCS_SECTEUR_X

1.5 Référence d'approbation et date d'effet

La publication demande obligatoirement une référence d'approbation et une date d'effet. Une version publiée avec une date future n'est utilisée qu'à partir de cette date.

Référence d'approbation : DEC-H/DT/2026-014
Date d'effet : 2026-08-01

Bonne pratique

La référence doit permettre de retrouver la décision, le procès-verbal, la note de service ou tout autre document officiel ayant autorisé la publication.

1.6 Règles d'écriture JSON

- Les clés et les textes sont placés entre guillemets doubles.
- Les éléments d'une liste sont séparés par des virgules.
- Les booléens s'écrivent true et false en minuscules.
- Les commentaires // ou /* */ ne sont pas autorisés dans le JSON.
- Une virgule finale après le dernier élément provoque une erreur.

JSON valide

```
{
  "active": true,
  "minimum": 1,
  "codes": ["A", "B", "C"]
}
```

JSON invalide : True et virgule finale

```
{  
  "active": True,  
  "minimum": 1,  
  "codes": ["A", "B", "C"],  
}
```

2

COLLECTE_COMPLETUDE

Définir les conditions de soumission d'une fiche de collecte.

2.1 Objectif métier

RM-35 autorise l'enregistrement d'une fiche incomplète au statut Brouillon, mais interdit sa soumission lorsque des informations obligatoires manquent. RM-20 demande le calcul automatique d'un indice de complétude fondé sur les champs définis par la HAUQE.

Référence : Règles métier validées RM-20 et RM-35.

Séparation essentielle

Le brouillon peut être incomplet. La règle COLLECTE_COMPLETUDE n'intervient comme verrou qu'au moment de la soumission.

2.2 Créer la règle dans l'interface

1 Ouvrir COLLECTE_COMPLETUDE

Dans Règles et codification, sélectionner l'onglet dédié.

2 Renseigner la version

Exemple : 1.0. La famille recommandée est COLLECTE.

3 Ajouter les exigences FIELD

Choisir les champs et le mode ALL ou ANY.

4 Ajouter les exigences COUNT

Ajouter si nécessaire documents, offres ou certifications déclarées.

5 Fixer le minimum

100 % impose que toutes les exigences soient satisfaites.

6 Valider la configuration

Le backend contrôle les noms de champs, les types et les valeurs.

7 Créer le brouillon

La règle est enregistrée sans être active.

8 Publier

Saisir la référence d'approbation et la date d'effet.

2.3 Structure JSON générale

```
{
  "requirements": [
    {
      "type": "FIELD",
      "label": "Exigence lisible",
      "fields": ["nom_du_champ"],
      "match": "ALL"
    }
  ],
  "minimum_submission_rate": 100
}
```

Clé	Type	Rôle
requirements	liste	Ensemble des exigences évaluées.
minimum_submission_rate	nombre 0 à 100	Pourcentage minimal d'exigences satisfaites.
type	FIELD ou COUNT	Nature de l'exigence.
label	texte	Nom affiché dans le diagnostic.
fields	liste de champs	Champs évalués pour FIELD.
match	ALL ou ANY	Tous les champs ou au moins un.
resource	code	Ressource comptée pour COUNT.
minimum	entier >= 1	Nombre minimal attendu.

2.4 Exigence FIELD avec ALL

Le mode ALL signifie que tous les champs listés doivent être renseignés. Pour un seul champ, ALL est le mode normal.

```
{
  "type": "FIELD",
  "label": "Identification de la fiche",
  "fields": [
    "entreprise_id",
    "version_formulaire"
  ],
  "match": "ALL"
}
```

2.5 Exigence FIELD avec ANY

Le mode ANY signifie qu'au moins un des champs doit être renseigné. C'est le cas typique d'un téléphone OU d'un courriel.

```
{
  "type": "FIELD",
  "label": "Contact du déclarant",
  "fields": [
    "telephone_declarant",
    "email_declarant"
  ],
  "match": "ANY"
}
```

2.6 Exigence COUNT

COUNT contrôle le nombre de ressources liées à la fiche. Le minimum doit être un entier supérieur ou égal à 1.

```
{
  "type": "COUNT",
  "label": "Au moins une pièce justificative",
  "resource": "DOCUMENTS",
  "minimum": 1
}
```

Ressource	Ce qui est compté
DOCUMENTS	Documents actifs rattachés à la ressource FICHE_COLLECTE.
OFFRES_DECLAREES	Offres déclarées liées à la fiche.
CERTIFICATIONS_DECLAREES	Certifications déclarées liées à la fiche.

2.7 Champs actuellement sélectionnables

Champ technique	Sens utilisateur	Remarque
entreprise_id	Entreprise liée	Identifiant de l'entreprise.
version_formulaire	Version du formulaire	Référence de la fiche utilisée.
consentement_obtenu	Consentement obtenu	Doit être true pour être satisfait.
nom_declarant	Nom du déclarant	Texte non vide.
fonction_declarant	Fonction du déclarant	Texte non vide.
telephone_declarant	Téléphone du déclarant	Peut être combiné en ANY.
email_declarant	Courriel du déclarant	Peut être combiné en ANY.
signature_declarant	Signature du déclarant	Référence ou valeur non vide.
observations	Observations	À rendre obligatoire uniquement sur décision.

Champs système exclus

mission_id, statut, taux_completude, numéro de révision, auteur et dates techniques ne sont pas sélectionnables comme exigences utilisateur.

2.8 Modèle complet à valider par la HAUQE

Ce modèle n'est pas auto-officiel

Il illustre la configuration. La HAUQE doit confirmer chaque exigence, notamment la signature et le document obligatoire.

```
{
  "requirements": [
    {
      "type": "FIELD",
      "label": "Entreprise et formulaire",
      "fields": [
        "entreprise_id",
        "version_formulaire"
      ],
      "match": "ALL"
    },
    {
      "type": "FIELD",
      "label": "Identité du déclarant",
      "fields": ["nom_declarant"],
      "match": "ALL"
    },
    {
      "type": "FIELD",
      "label": "Contact du déclarant",
      "fields": [
        "telephone_declarant",
        "email_declarant"
      ],
      "match": "ANY"
    },
    {
      "type": "FIELD",
      "label": "Consentement",
      "fields": ["consentement_obtenu"],
      "match": "ALL"
    },
    {
      "type": "COUNT",
      "label": "Pièce justificative",
      "resource": "DOCUMENTS",
      "minimum": 1
    }
  ],
  "minimum_submission_rate": 100
}
```

2.9 Comment le taux est calculé

$$\text{taux} = \frac{\text{exigences satisfaites} \times 100}{\text{nombre d'exigences}}$$

Chaque bloc FIELD ou COUNT compte comme une exigence. Une exigence ANY avec deux champs vaut une seule exigence : elle est satisfaite si au moins un champ est présent.

Seuil inférieur à 100 %

Un minimum de 80 % autorise la soumission avec certaines exigences non satisfaites. Ce choix doit être explicite et approuvé, car il réduit la force du verrou RM-35.

2.10 Test de recette

Cas	Résultat attendu
Aucune règle publiée	Soumission bloquée par sécurité.
Règle invalide publiée	Erreur de configuration, soumission bloquée.
Exigence manquante et minimum 100 %	Soumission refusée.
Toutes les exigences satisfaites	Soumission autorisée.
Version future publiée	Inactive jusqu'à sa date d'effet.

3

Règles métier génériques

Créer des paramètres JSON consommés par les moteurs HAUQE.

3.1 Quand utiliser une règle générique ?

Une règle générique convient à un moteur qui lit un objet JSON versionné : seuils d'alerte, délais, listes, conditions ou paramètres de décision. Elle ne remplace pas les modules spécialisés comme les modèles de scoring ou les grilles FUCCS.

Effet réel

Publier un code inconnu ne crée pas automatiquement un comportement. Le moteur concerné doit être programmé pour résoudre ce code logique et interpréter son JSON.

3.2 Champs à renseigner

Champ	Exemple	Règle
Code logique	VEILLE_SEUILS_EXPIRATION	Stable entre les versions.
Famille	VEILLE	Classement fonctionnel.
Libellé	Seuils d'expiration	Compréhensible par l'utilisateur.
Version	1.0	Édition institutionnelle.
Description	Seuils RM-05 à RM-08	But et périmètre.
Paramètres JSON	{...}	Structure attendue par le moteur.
Date d'effet	2026-08-01	Peut être fixée à la publication.

3.3 Exemple validé : VEILLE_SEUILS_EXPIRATION

Les règles RM-05 à RM-08 définissent quatre seuils : 180 jours, 90 jours, 30 jours et expiration. Le moteur de veille accepte la structure suivante.

```
{
  "thresholds": [
    {
      "days": 180,
      "niveau": 1,
      "code": "INFO_180",
      "label": "Information"
    },
    {
      "days": 90,
      "niveau": 2,
      "code": "SURVEILLANCE_90",
      "label": "Surveillance"
    },
    {
      "days": 30,
      "niveau": 3,
      "code": "URGENCE_30",
      "label": "Urgence"
    },
    {
      "days": 0,
      "niveau": 4,
      "code": "CRITIQUE_EXPIRATION",
      "label": "Critique"
    }
  ]
}
```

Référence : Règles métier validées RM-05 à RM-08 et moteur Veille HAUQE Certif.

3.4 Créer une nouvelle version

1 Sélectionner la version publiée

La consulter sans la modifier.

2 Cliquer Nouvelle version

Le clone reprend le code logique et les paramètres.

3 Changer la version

Exemple : 1.0 -> 1.1.

4 Modifier le JSON

Ne changer que les paramètres approuvés.

5 Valider et tester

Comparer le résultat avec la version actuelle.

6 Publier

La nouvelle date d'effet détermine le basculement.

3.5 Erreurs JSON fréquentes

Erreur	Symptôme	Correction
Virgule finale	JSON impossible à enregistrer.	Supprimer la dernière virgule.
Clé mal orthographiée	Le moteur ignore ou refuse le paramètre.	Utiliser exactement la clé documentée.
Nombre écrit comme texte	Comparaison ou calcul incorrect.	Utiliser 180 et non "180".
Booléen True/False	JSON invalide.	Utiliser true/false.
Code logique modifié	Nouvelle famille créée par erreur.	Cloner en conservant le code logique.

4

Modèles de scoring et INFC

Configurer les formules, les classes, les niveaux et les pondérations.

4.1 Règle métier ou modèle de scoring ?

Objet	Stockage	Usage
Règle générique	regles_metier.parametres	Seuils, délais, listes, conditions.
Modèle de scoring	modeles_scoring.regle_calcul	Formule, arrondi, classes ou niveaux.
Pondération	ponderations_scoring	Poids ou maximum de points par domaine.
Grille FUCCS	grilles/rubriques/criteres_fuccs	Contrôle critère par critère.

4.2 Objets évalués disponibles

- CLASSIFICATION_ENTREPRISE : produit une classe globale de l'entreprise.
- INFC : produit un score global et un niveau pour une certification.

Ne pas mélanger

Classification entreprise, INFC, FUCCS et SNCC sont des résultats distincts. Aucun passage automatique FUCCS -> INFC n'est supposé.

4.3 Modes de calcul supportés

Mode	Entrée	Calcul
DIRECT_SCORE	Un score déjà calculé.	Le modèle applique les classes ou niveaux.
WEIGHTED_AVERAGE_100	Chaque domaine noté de 0 à 100.	Moyenne pondérée sur 100.
SUM_DOMAIN_POINTS	Chaque domaine noté entre 0 et son maximum.	Somme des points obtenus.

WEIGHTED_AVERAGE_100

```
score pondéré = somme(score_domaine x poids)
                -----
                somme(poids)
```

SUM_DOMAIN_POINTS

```
score global = somme(points obtenus par domaine)
```

4.4 Clés communes de regle_calcul

Clé	Valeur	Utilité
calculation_mode	DIRECT_SCORE, WEIGHTED_AVERAGE_100 ou SUM_DOMAIN_POINTS	Mode obligatoire.
rounding	0 à 6	Nombre de décimales.
score_min	nombre	Borne minimale autorisée.
score_max	nombre	Borne maximale autorisée.
missing_policy	REJECT	Bloque le calcul si un domaine manque.
classes	liste	Classification entreprise.
levels	liste	Niveaux INFC.

4.5 Classification entreprise RM-22 à RM-24

Les seuils validés sont : Conforme de 85 à 100, À surveiller de 60 à moins de 85, Non conforme sous 60. Le moteur teste les classes dans l'ordre et accepte une classe par défaut.

```
{
  "calculation_mode": "DIRECT_SCORE",
  "rounding": 2,
  "score_min": 0,
  "score_max": 100,
  "classes": [
    {
      "code": "CONFORME",
      "min": 85
    },
    {
      "code": "A_SURVEILLER",
      "min": 60
    },
    {
      "code": "NON_CONFORME",
      "default": true
    }
  ]
}
```

Référence : Règles métier validées RM-22, RM-23 et RM-24.

Ordre des classes

CONFORME doit apparaître avant A_SURVEILLER. Sinon un score de 90 pourrait être capturé par la règle min 60.

4.6 Créer le modèle CLASSIFICATION_ENTREPRISE

1

Nouveau modèle

Choisir objet CLASSIFICATION_ENTREPRISE.

2 Code et version

Code recommandé CLASSIFICATION_ENTREPRISE, version 1.0.

3 Mode DIRECT_SCORE

Le score global est fourni à l'évaluation.

4 Coller les classes

Utiliser le JSON RM-22 à RM-24.

5 Créer le brouillon

Le modèle reste inactif.

6 Tester la prévisualisation

Essayer 90, 70 et 50.

7 Publier

Référence d'approbation et date de début.

Score test	Classe attendue
90	CONFORME
70	A_SURVEILLER
50	NON_CONFORME

4.7 Pondérations

Les pondérations sont saisies dans la section dédiée du modèle, et non dans le tableau classes/levels. Leur sens dépend du mode de calcul.

Mode	Sens de valeur
DIRECT_SCORE	Aucune pondération nécessaire pour le calcul direct.
WEIGHTED_AVERAGE_100	Poids relatif du domaine.
SUM_DOMAIN_POINTS	Maximum de points attribuable au domaine.

Total des pondérations

Pour SUM_DOMAIN_POINTS, la somme représente le score maximal théorique. Pour WEIGHTED_AVERAGE_100, la somme peut être 100, mais le moteur normalise aussi une autre somme positive.

4.8 INFC : modèle en attente de décision finale

La structure actuelle documente six domaines pondérés sur 100. Cependant, les sources de chaque domaine, la formule opérationnelle et la traduction numérique des niveaux doivent être approuvées avant publication définitive.

Domaine technique	Maximum documenté
AUTHENTICITE	20
VALIDITE	20
MAINTIEN	20
MAITRISE_DOCUMENTAIRE	15
TRACABILITE_MAITRISE_OPERATIONNELLE	15
SUIVI_RENOUVELLEMENT	10

Gabarit de travail - ne pas publier tel quel

```
{
  "calculation_mode": "SUM_DOMAIN_POINTS",
  "rounding": 2,
  "score_min": 0,
  "score_max": 100,
  "missing_policy": "REJECT",
  "levels": [
    {
      "niveau": 1,
      "min": "A_DEFINIR",
      "max": "A_DEFINIR"
    }
  ]
}
```

Blocage volontaire

Les textes A_DEFINIR ne sont pas des valeurs valides. Ils servent à empêcher une publication prématurée. Remplacer par des nombres uniquement après décision institutionnelle.

4.9 Politique des données manquantes

La valeur recommandée est REJECT : si un domaine requis manque, le calcul est bloqué. Aucune imputation silencieuse n'est effectuée.

```
{
  "missing_policy": "REJECT"
}
```

4.10 Recette d'un modèle de scoring

Contrôle	Attendu
Mode connu	DIRECT_SCORE, WEIGHTED_AVERAGE_100 ou SUM_DOMAIN_POINTS.
Arrondi	Entier de 0 à 6.
Bornes	Tous les scores de test appartiennent à une classe/niveau.
Domaines	Pas de domaine manquant si REJECT.
Pondérations	Valeurs positives et cohérentes avec le mode.
Publication	Référence et date de début renseignées.

5

Grilles de contrôle FUCCS

Créer une grille versionnée, ses rubriques et ses critères.

5.1 Structure fonctionnelle

```
FUCCS (grille, version 1.0)
|
+-- FUCCS_R01 (rubrique)
|   +-- FUCCS_R01_C01 (critère)
|   +-- FUCCS_R01_C02
|
+-- FUCCS_R02
|   +-- FUCCS_R02_C01
```

La version appartient à la grille. Les rubriques et critères conservent leurs codes lors du clonage afin de comparer les résultats entre versions.

5.2 Créer une grille brouillon

1 Ouvrir Grilles FUCCS

La liste présente brouillons, versions publiées et retirées.

2 Nouvelle grille

Saisir code, version, libellé et date prévue.

3 Code recommandé

Utiliser FUCCS pour une grille nationale unique.

4 Ajouter les rubriques

Créer FUCCS_R01, FUCCS_R02, etc.

5 Ajouter les critères

Créer FUCCS_R01_C01, FUCCS_R01_C02, etc.

6 Contrôler le total

Vérifier le score maximal calculé et les poids renseignés.

7 Publier

Référence d'approbation et date d'effet.

5.3 Codification de la grille

Niveau	Code	Exemple
Grille	<FAMILLE>	FUCCS
Rubrique	<GRILLE>_Rnn	FUCCS_R01
Critère	<RUBRIQUE>_Cnn	FUCCS_R01_C01
Version	champ séparé	1.0

- La génération automatique s'appuie sur l'ordre, mais le code peut être ajusté avant enregistrement.
- Le code reste stable même si l'ordre d'affichage change.
- Le clonage copie les mêmes codes dans la nouvelle version.
- La non-réutilisation d'un code supprimé est une règle de gouvernance à respecter.

5.4 Créer une rubrique

Champ	Obligatoire	Conseil
Ordre	Oui	Numéro d'affichage, distinct du code.
Code	Oui	FUCCS_R01, FUCCS_R02...
Libellé	Oui	Intitulé fonctionnel clair.
Description	Non	But et périmètre de la rubrique.

5.5 Créer un critère

Champ	Effet dans le contrôle
Code	Identifie le critère dans l'historique et les notes.
Libellé / description	Explique ce qui doit être vérifié.
Ordre	Détermine l'affichage dans la rubrique.
Score maximal	Limite la note et contribue au score maximal global.
Poids	Information paramétrique stockée ; le calcul FUCCS actuel additionne les scores maximaux.
Commentaire obligatoire	Le contrôle ne peut être finalisé sans commentaire.
Preuve obligatoire	Le contrôle ne peut être finalisé sans document actif.

Poids FUCCS

Dans le moteur actuel, le taux FUCCS est score brut / somme des scores maximaux. Le champ poids est conservé pour paramétrage et évolution, mais ne modifie pas ce calcul tant qu'une règle spécifique n'est pas implémentée.

5.6 Conditions de publication

Condition	Contrôle backend
Au moins une rubrique	Obligatoire.
Au moins un critère	Obligatoire.
Score maximal total > 0	Obligatoire.
Référence d'approbation	Obligatoire.
Date d'effet	Obligatoire.

Ce qui n'est pas imposé

Le backend ne force ni 24 critères, ni 28 critères, ni un total de 100 points. Ces choix doivent être portés par la grille approuvée.

5.7 Effet des obligations sur le contrôle

Lors de la notation, un score ne peut pas dépasser le score maximal du critère. Si commentaire_obligatoire ou preuve_obligatoire vaut true, la finalisation du contrôle est bloquée tant que l'élément manque.

```
Critère : FUCCS_R01_C01
Score maximal : 5
Commentaire obligatoire : true
Preuve obligatoire : true

Résultat : note <= 5 + commentaire + preuve active requis
```

5.8 Cloner une nouvelle version

Le clonage copie la grille, les rubriques, les critères, les scores, les poids et les obligations dans un nouveau brouillon.

```
FUCCS v1.0 publié
FUCCS_R01
FUCCS_R01_C01

Clonage vers FUCCS v1.1 brouillon
FUCCS_R01
FUCCS_R01_C01
```

Comparabilité

Conserver les mêmes codes lorsque le sens du critère reste le même. Créer un nouveau code lorsque le critère change de nature.

5.9 Retirer une grille

Seule une grille publiée peut être retirée. La date de fin ne peut pas être antérieure à la date d'effet. Le retrait n'efface pas la grille ni les contrôles historiques qui la référencent.

5.10 Recette FUCCS avant publication

Vérification	Question
Structure	Toutes les rubriques prévues existent-elles ?
Codes	Les codes sont-ils uniques, stables et compréhensibles ?
Critères	Chaque critère est-il clair et mesurable ?
Scores	Les maximums sont-ils cohérents avec le total attendu ?
Obligations	Les preuves/commentaires obligatoires sont-ils justifiés ?
Test	Un contrôle complet peut-il être saisi et finalisé ?
Approbation	La référence officielle et la date d'effet sont-elles connues ?

6

Checklists et dépannage

Publier proprement et corriger les erreurs les plus fréquentes.

6.1 Checklist universelle avant publication

- Le code logique correspond exactement au moteur consommateur.
- La version est nouvelle et cohérente avec l'impact du changement.
- Le JSON est valide et ne contient aucune valeur A_DEFINIR.
- Les seuils, bornes et listes couvrent tous les cas de test.
- Les pondérations sont positives et cohérentes avec le mode de calcul.
- La configuration a été testée dans un environnement de recette.
- La référence d'approbation est officielle et retrouvable.
- La date d'effet est correcte et ne provoque pas un basculement prématuré.
- Les utilisateurs concernés ont été informés du changement.

6.2 Messages fréquents

Message ou symptôme	Cause probable	Action
Aucune règle publiée/active	Aucune version PUBLIE applicable.	Publier ou vérifier la date d'effet.
Règle invalide	JSON mal formé ou champ inconnu.	Lancer la validation et corriger le détail.
Permission insuffisante	Rôle sans permission d'administration.	Faire vérifier le RBAC.
Version déjà existante	Même code et même version.	Choisir une nouvelle version.
Score hors classe	Bornes incomplètes ou ordre incorrect.	Revoir classes/levels.
Domaine manquant	missing_policy=REJECT.	Fournir le domaine ou revoir la règle.
Grille vide	Aucune rubrique/critère ou score total nul.	Compléter le brouillon.
Grille immuable	Version publiée ou retirée.	Cloner une nouvelle version.

6.3 Que faire en cas d'erreur JSON ?

1

Ne pas publier

Conserver la version au statut BROUILLON.

2

Copier le message

Noter la clé ou l'exigence mentionnée.

3 Vérifier la syntaxe

Guillemets, virgules, crochets, accolades.

4 Vérifier le schéma

Comparer les clés au présent guide.

5 Valider à nouveau

La réponse normalisée doit être correcte.

6 Tester un cas simple

Confirmer le résultat avant publication.

6.4 Retour arrière

Une version publiée ne doit pas être éditée. En cas d'erreur après publication :

- Créer immédiatement une nouvelle version corrigée par clonage.
- La tester et la publier avec une nouvelle référence ou un avenant à la décision.
- Retirer la version erronée avec une date et un motif.
- Conserver toutes les traces dans le journal d'audit.

6.5 Glossaire

Terme	Définition
Code logique	Identifiant fonctionnel stable d'une règle ou d'un modèle.
Code physique	Identifiant technique unique d'une version de règle générique.
Brouillon	Version modifiable sans effet opérationnel.
Publié	Version immuable utilisée selon sa date d'effet.
Retiré	Version historisée qui n'est plus sélectionnée pour de nouveaux traitements.
Paramètres JSON	Structure de données interprétée par un moteur.
Pondération	Poids relatif ou maximum de points d'un domaine.
Readiness	État indiquant si les paramètres nécessaires sont publiés.
FUCCS	Grille de contrôle versionnée composée de rubriques et critères.

A

Annexes JSON prêtes à copier

Utiliser uniquement après validation du contenu institutionnel.

A.1 - Squelette de règle générique

```
{
  "parametre_1": "VALEUR",
  "liste": [
    {
      "code": "CODE_1",
      "label": "Libellé 1"
    }
  ]
}
```

A.2 - COLLECTE_COMPLETUDE minimal

```
{
  "requirements": [
    {
      "type": "FIELD",
      "label": "Entreprise liée",
      "fields": ["entreprise_id"],
      "match": "ALL"
    }
  ],
  "minimum_submission_rate": 100
}
```

A.3 - COLLECTE_COMPLETUDE avec alternative et document

```
{
  "requirements": [
    {
      "type": "FIELD",
      "label": "Contact du déclarant",
      "fields": [
        "telephone_declarant",
        "email_declarant"
      ],
      "match": "ANY"
    },
    {
      "type": "COUNT",
      "label": "Document actif",
      "resource": "DOCUMENTS",
      "minimum": 1
    }
  ],
  "minimum_submission_rate": 100
}
```

A.4 - Classification entreprise RM-22 à RM-24

```
{
  "calculation_mode": "DIRECT_SCORE",
  "rounding": 2,
  "score_min": 0,
  "score_max": 100,
  "classes": [
    {"code": "CONFORME", "min": 85},
    {"code": "A_SURVEILLER", "min": 60},
    {"code": "NON_CONFORME", "default": true}
  ]
}
```

A.5 - Modèle pondéré générique

```
{
  "calculation_mode": "WEIGHTED_AVERAGE_100",
  "rounding": 2,
  "score_min": 0,
  "score_max": 100,
  "missing_policy": "REJECT",
  "classes": [
    {"code": "CLASSE_HAUTE", "min": 80},
    {"code": "CLASSE_BASSE", "default": true}
  ]
}
```

À compléter dans l'interface

Les domaines et leurs pondérations sont ajoutés dans la section Pondérations du modèle. Ils ne sont pas placés dans ce JSON.

A.6 - Seuils de veille RM-05 à RM-08

```
{
  "thresholds": [
    {"days": 180, "niveau": 1, "code": "INFO_180", "label": "Information"},
    {"days": 90, "niveau": 2, "code": "SURVEILLANCE_90", "label": "Surveillance"},
    {"days": 30, "niveau": 3, "code": "URGENCE_30", "label": "Urgence"},
    {"days": 0, "niveau": 4, "code": "CRITIQUE_EXPIRATION", "label": "Critique"}
  ]
}
```

A.7 - Fiche de décision avant publication

Information	À renseigner
Objet	Règle / modèle / grille concernée
Code logique	_____
Version	_____
Résumé du changement	_____
Cas de test réalisés	_____
Résultats attendus	_____

Information	À renseigner
Référence d'approbation	_____
Date d'effet	_____
Responsable de publication	_____

Sources du guide

- Règles métier validation GFA - version améliorée : RM-05 à RM-08, RM-20, RM-22 à RM-24, RM-35 et RM-51.
- SNCC HAUQE version 1.0 : distinction entre conformité, statut administratif et risque.
- Schémas et services HAUQE Certif : gouvernance, scoring, collecte et FUCCS, état du 28 juillet 2026.

Fin du guide

Après chaque publication, actualiser la page et vérifier les cartes de readiness avant de reprendre les tests Collecte, Scoring ou FUCCS.